

Utilizando a API OpenGL e as funções **glPushMatrix()** e **glPopMatrix()**, o código aplica transformações de translação, rotação e escala de forma hierárquica. Cada parte do robô é desenhada utilizando primitivas geométricas (neste caso, o **glutSolidCube()** para facilitar a modelagem), e as animações são controladas por uma função de timer que atualiza os ângulos de rotação.

Resultados:

A execução do programa resulta em uma cena 3D onde o robô exibe os seguintes movimentos:

- O corpo gira lentamente, criando uma visão dinâmica do sistema global.
- O braço oscila suavemente (de -45° a 45°) em relação ao corpo.
- O antebraço realiza uma rotação contínua em torno de sua extremidade, demonstrando a transmissão de transformações através da hierarquia.

```
C/C++
// robot_animado.c
#include <GL/glut.h>
#include <math.h>

// Variáveis globais para controlar a animação
float t = 0.0;           // tempo (ou contador)
float bodyAngle = 0.0;   // Ângulo de rotação do corpo (eixo Y)
float armAngle = 0.0;    // Ângulo de oscilação do braço (eixo Z)
float forearmAngle = 0.0; // Ângulo de rotação do antebraço (em torno de
                          // seu ponto de ancoragem)

// Função de exibição
void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    // Configuração da câmera
    gluLookAt(0.0, 2.0, 10.0, // posição da câmera
              0.0, 0.0, 0.0,  // para onde a câmera aponta
              0.0, 1.0, 0.0); // vetor "up"

    // Início do sistema global: Corpo do robô
    glPushMatrix();
    // O corpo gira lentamente em torno do eixo Y
    glRotatef(bodyAngle, 0.0, 1.0, 0.0);
```

```

// Desenha o corpo (um cubo alongado)
glPushMatrix();
    glColor3f(0.7, 0.7, 0.7); // cinza
    glScalef(0.8, 1.2, 0.4); // escala para dar forma de corpo
    glutSolidCube(1.0);
glPopMatrix();

// Início do sistema local: Braço
glPushMatrix();
    // Posiciona o braço na lateral direita do corpo
    glTranslatef(0.5, 0.3, 0.0);
    // O braço oscila de um lado para o outro em torno do eixo Z
    glRotatef(armAngle, 0.0, 0.0, 1.0);
    // Posiciona o braço: desloca para que a rotação ocorra na
extremidade
    glTranslatef(0.75, 0.0, 0.0);
    // Desenha o braço (um retângulo alongado)
    glPushMatrix();
        glColor3f(1.0, 0.0, 0.0); // vermelho
        glScalef(1.5, 0.3, 0.3); // dimensões do braço
        glutSolidCube(1.0);
    glPopMatrix();

    // Início do sistema local para o antebraço
    glPushMatrix();
        // Move a origem para a extremidade do braço
        glTranslatef(0.75, 0.0, 0.0);
        // O antebraço gira em torno da sua extremidade
        glRotatef(forearmAngle, 0.0, 0.0, 1.0);
        // Posiciona o antebraço para que a rotação ocorra no seu
centro
        glTranslatef(0.75, 0.0, 0.0);
        // Desenha o antebraço (um retângulo)
        glColor3f(0.0, 0.0, 1.0); // azul
        glPushMatrix();
            glScalef(1.5, 0.25, 0.25);
            glutSolidCube(1.0);
        glPopMatrix();
    glPopMatrix();
    // Fim do antebraço
glPopMatrix();
// Fim do braço
glPopMatrix();
// Fim do corpo do robô

glutSwapBuffers();
}

```

```

// Função de atualização (timer)
void timer(int value) {
    t += 1.0;
    // Atualiza o ângulo do corpo (rotaciona lentamente)
    bodyAngle = fmod(t * 0.5, 360.0);
    // Atualiza o ângulo do braço: oscila entre -45° e 45° (usando função
    seno)
    armAngle = 45.0 * sin(t * 0.05);
    // Atualiza o ângulo do antebraço (rotaciona continuamente)
    forearmAngle = fmod(t * 3.0, 360.0);

    glutPostRedisplay();
    glutTimerFunc(16, timer, 0); // Aproximadamente 60 FPS
}

// Função de inicialização do OpenGL
void init() {
    glEnable(GL_DEPTH_TEST);
    glClearColor(0.0, 0.0, 0.0, 1.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45.0, 1.33, 0.1, 100.0);
    glMatrixMode(GL_MODELVIEW);
}

// Função principal
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Robô com Braço Animado");

    init();
    glutDisplayFunc(display);
    glutTimerFunc(0, timer, 0);
    glutMainLoop();
    return 0;
}

```